TP1

Application Monolithique avec Spring Boot

Gestion de produits • API REST CRUD • Architecture en couches

Réalisé par : Nabila Abouilaaz

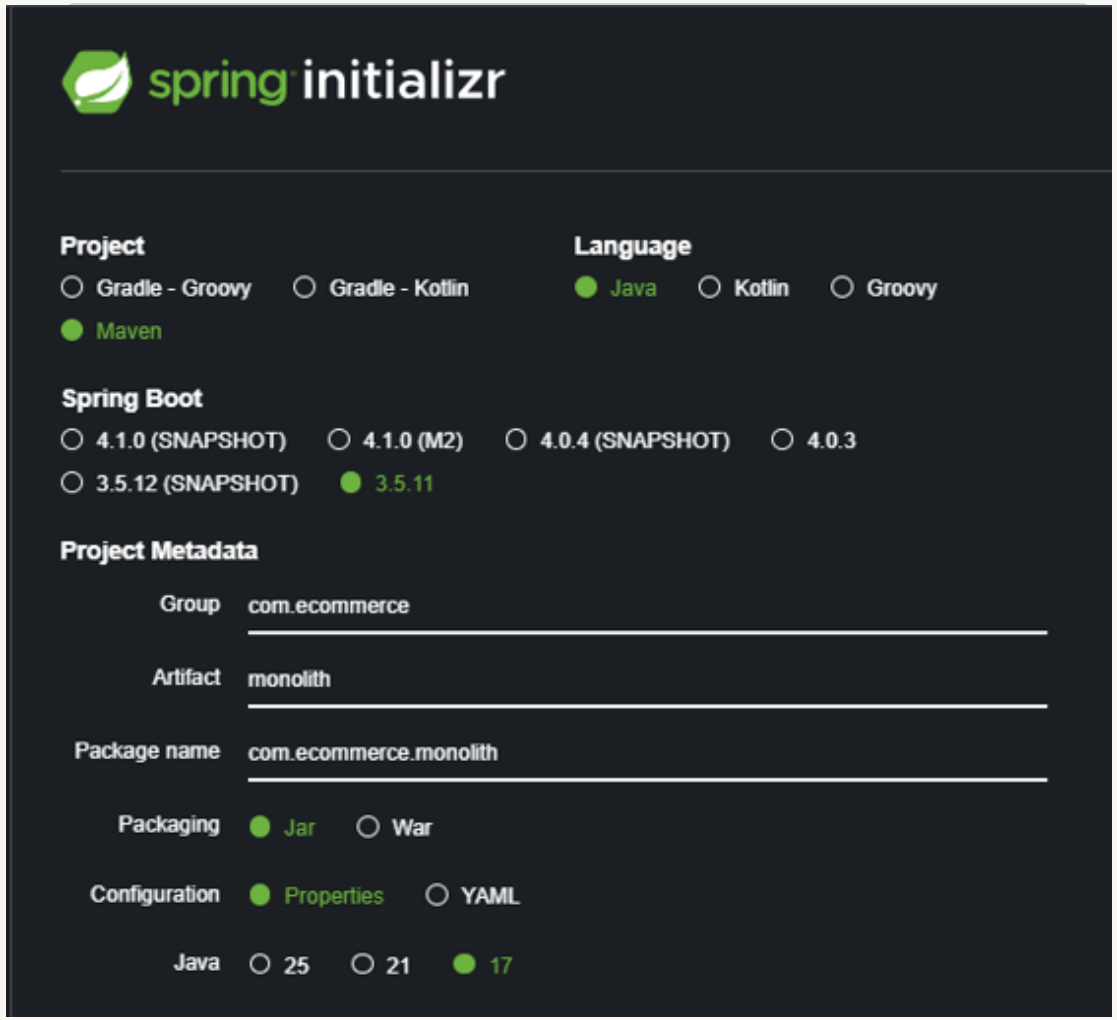
Encadré par : Pr. BE ELBAGHAZAOUI

Objectif du TP et livrables attendus

Ce TP consiste à développer une application monolithique simple avec Spring Boot pour gérer des produits via une API REST complète.

- Comprendre l'architecture en couches : Entity, Repository, Service, Controller.
- Configurer une base de données relationnelle et connecter Spring Data JPA.
- Implémenter les opérations CRUD et les valider avec Postman.
- Étendre ensuite le modèle avec Category et Spring Data REST.

5	4	2
Endpoints CRUD	Couches métier	Entités finales



Technologies utilisées et configuration initiale

Backend

Java 17
Spring Boot
Spring Web

Persistance

Spring Data JPA
PostgreSQL ou MySQL

Outils

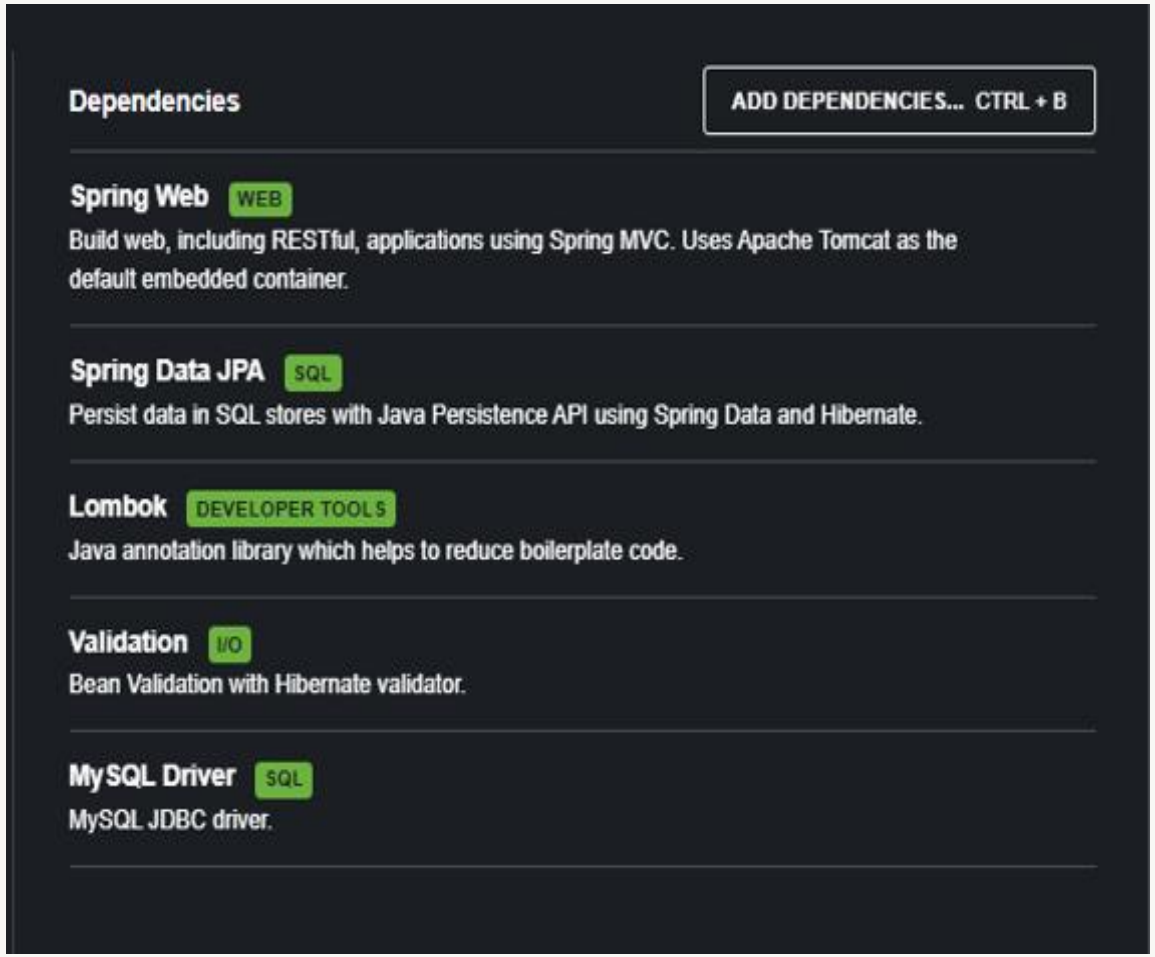
Maven
IntelliJ IDEA
Postman

Qualité

Validation
Lombok
Tests manuels API

Points de configuration :

- URL de connexion à la base de données
- username / password
- spring.jpa.hibernate.ddl-auto=update
- server.port=8080



Structure du projet et architecture en couches

model

Contient les entités JPA représentant les tables de la base de données.

repository

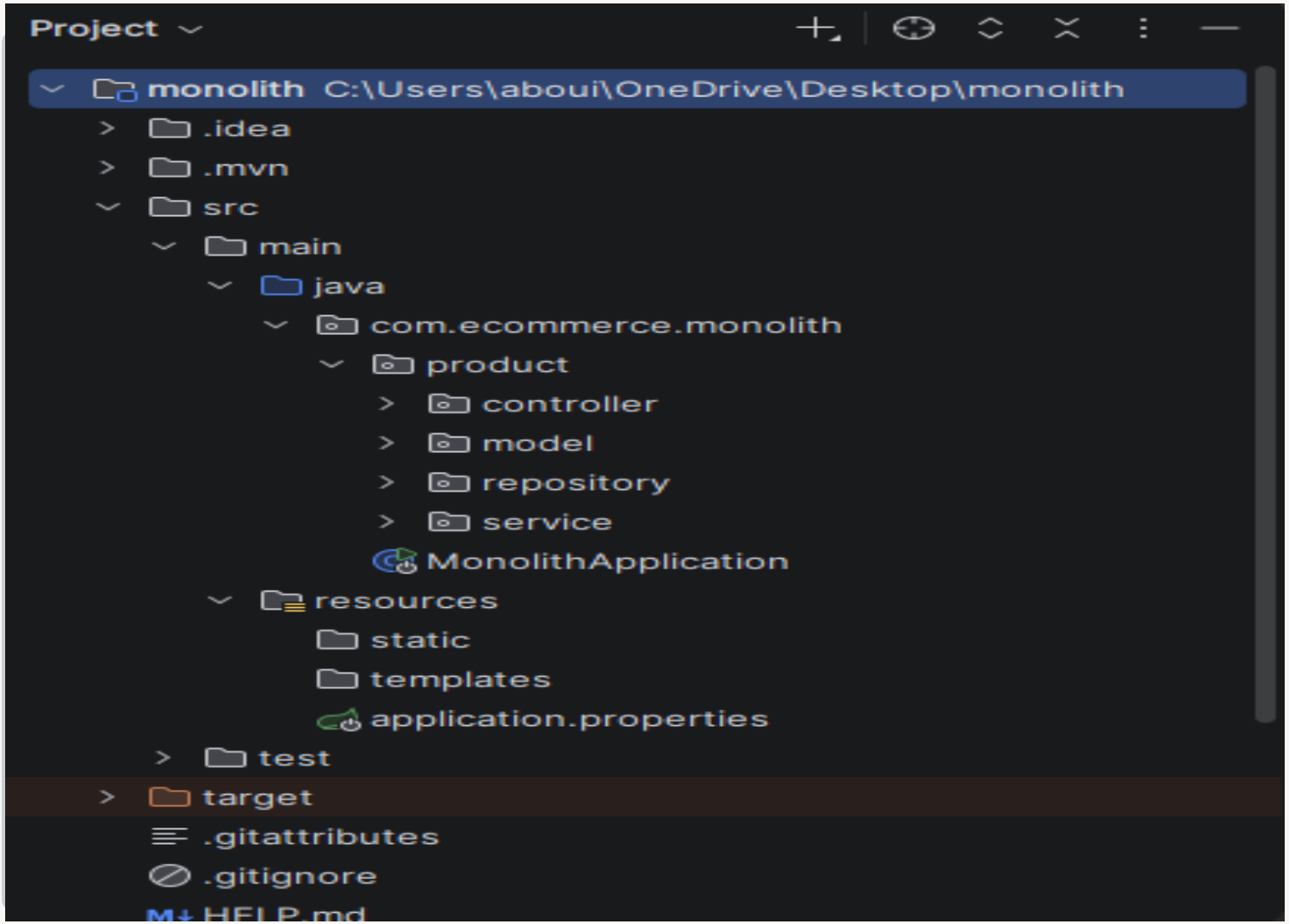
Assure l'accès aux données via JpaRepository et les opérations de persistance.

service

Centralise la logique métier : création, modification, suppression et consultation.

controller

Expose les endpoints REST et gère les échanges HTTP avec le client.



Entité Product et modélisation des données

L'entité Product représente la table des produits. Elle contient les informations nécessaires au CRUD et applique des validations simples.

- id : identifiant unique généré automatiquement
- name : nom du produit (obligatoire)
- description : description détaillée
- price : prix strictement positif
- stock : quantité disponible, positive ou nulle

Annotations importantes : @Entity, @Table, @Id, @GeneratedValue, @NotBlank, @NotNull, @Positive, @PositiveOrZero

application.properties | Product.java | ProductRepository.java

```java
package com.ecommerce.monolith.product.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;

import java.math.BigDecimal;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
```

application.properties | Product.java | ProductRepository.java

```java
@Builder
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
}
```

Implémentation du backend : Repository, Service et Controller

ProductRepository

Hérite de JpaRepository<Product, Long> et fournit automatiquement findAll(), findById(), save() et delete().

ProductService

Regroupe la logique métier : récupérer tous les produits, chercher par id, créer, mettre à jour et supprimer.

ProductController

Expose l'API REST sous /api/products et retourne des ResponseEntity adaptées à chaque opération.

application.properties | Product.java | ProductRepository.java | Trial

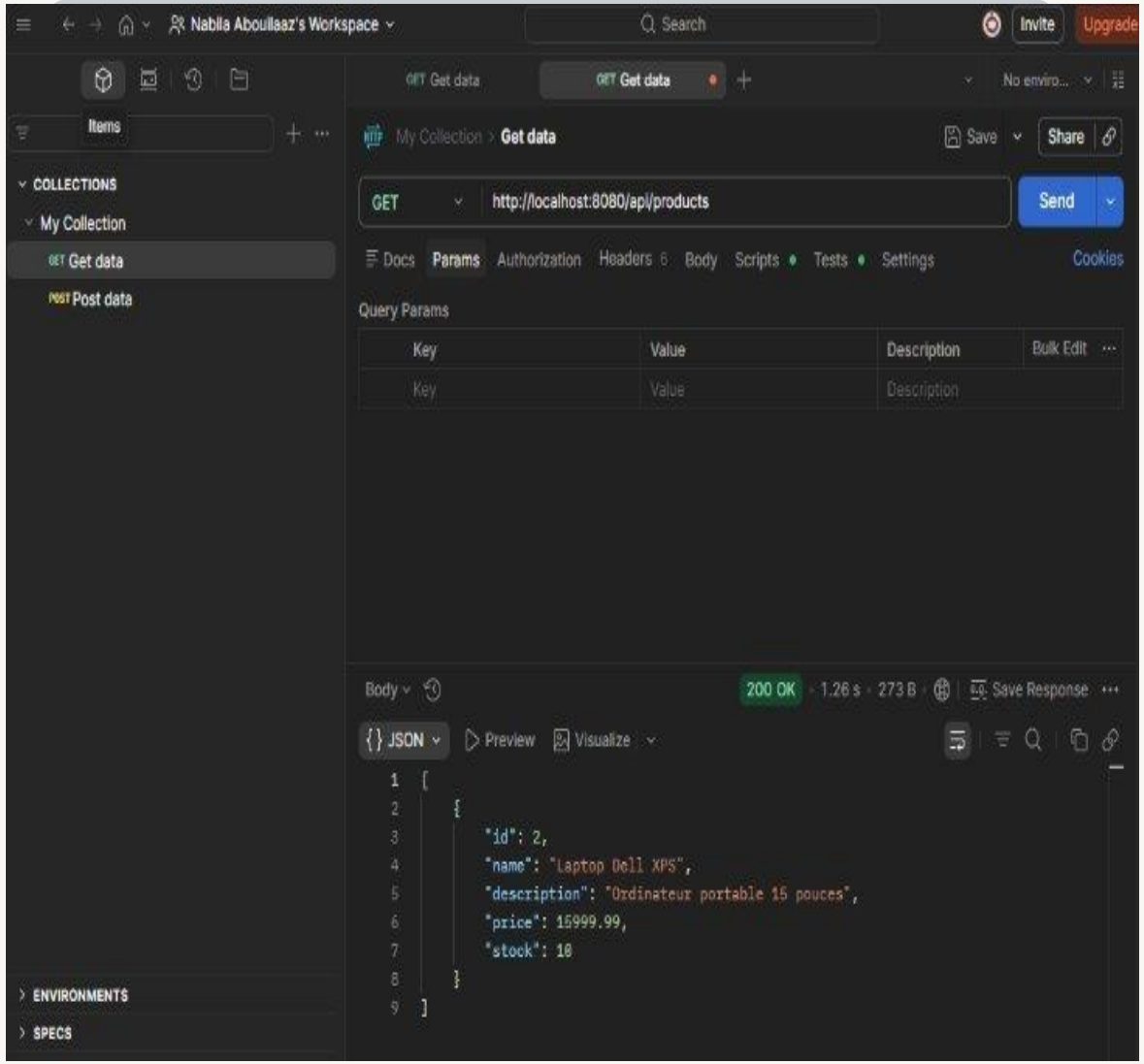
```java
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository no usages
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByNameContainingIgnoreCase(String name); no usages
}
```

application.properties | Product.java | ProductService.java | ProductController.java

```java
package com.ecommerce.monolith.product.controller;

import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.service.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService service;

    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        return ResponseEntity.ok(service.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
```

API REST CRUD développée

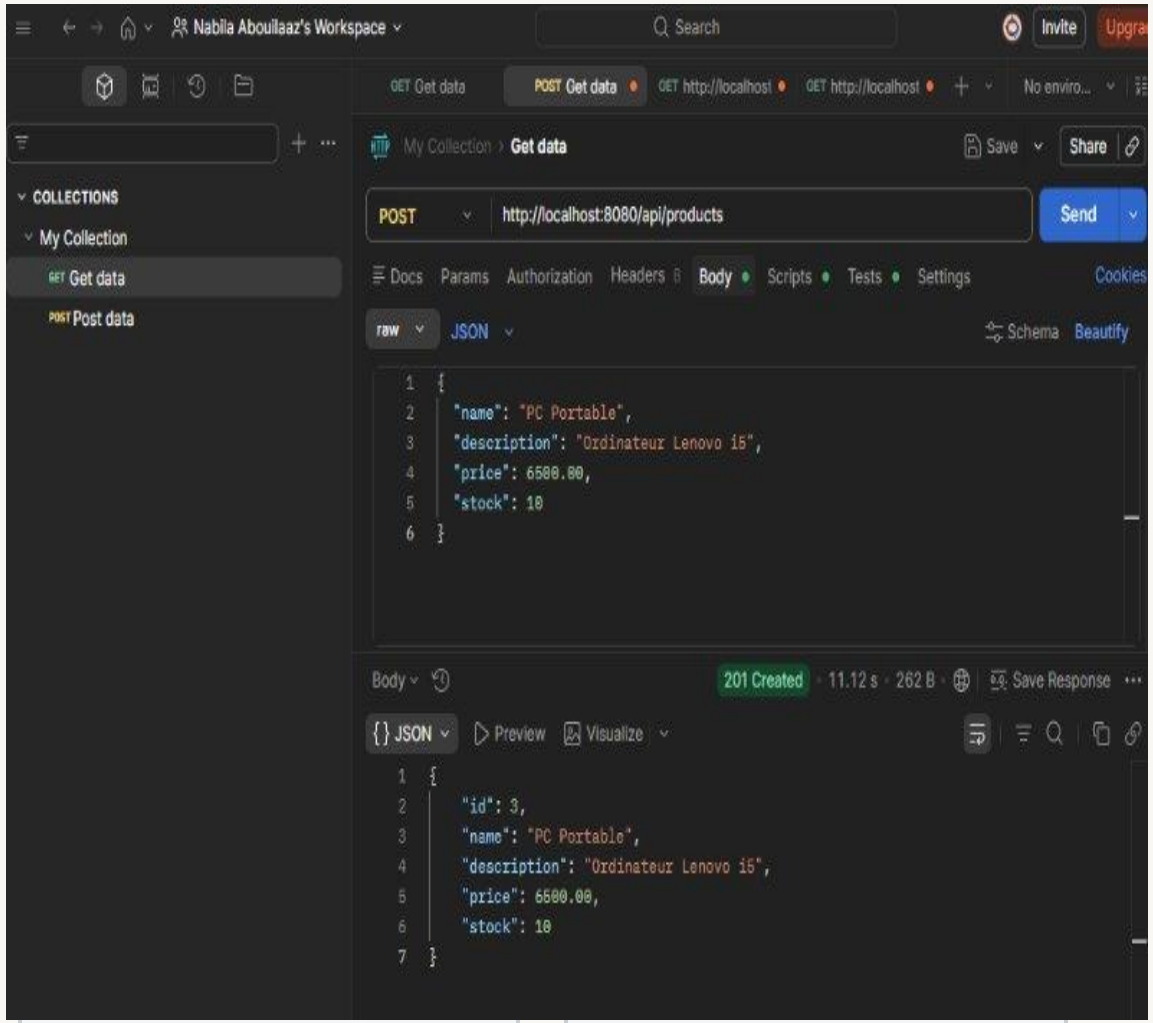
Méthode	Endpoint	Rôle	Réponse
GET	/api/products	Lister tous les produits	200 OK
GET	/api/products/{id}	Consulter un produit	200 OK
POST	/api/products	Créer un nouveau produit	201 Created
PUT	/api/products/{id}	Modifier un produit	200 OK
DELETE	/api/products/{id}	Supprimer un produit	204 No Content



Tests réalisés avec Postman

Les tests ont validé le comportement attendu de l'API ainsi que la bonne connexion avec la base de données.

- GET /api/products : récupération de la liste des produits
- POST /api/products : insertion d'un nouveau produit
- GET /api/products/{id} : consultation d'un produit précis
- PUT /api/products/{id} : mise à jour des données
- DELETE /api/products/{id} : suppression du produit



Exercice complémentaire : Category et relation ManyToOne

Afin d'étendre le modèle, une entité Category a été ajoutée pour regrouper les produits par catégorie.

- Une catégorie peut contenir plusieurs produits.
- Chaque produit appartient à une seule catégorie.
- La relation est modélisée dans Product avec @ManyToOne et @JoinColumn(name = "category_id").
- La table products contient donc une clé étrangère vers categories

Entité Category

id, name, description

Clé étrangère

products.category_id

MonolithApplication.java | pom.xml (monolith) | Category.java

```java
package com.ecommerce.monolith.product.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import lombok.*;

@Entity
@Table(name = "categories")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Category name is required")
    @Column(nullable = false, unique = true)
    private String name;

    @Column(length = 500)
```

n.java | pom.xml (monolith) | Category.java | Product.java

```java
public class Product {

    @Column(length = 1000)
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;

    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;
}
```

Spring Data REST et liaison produit-catégorie

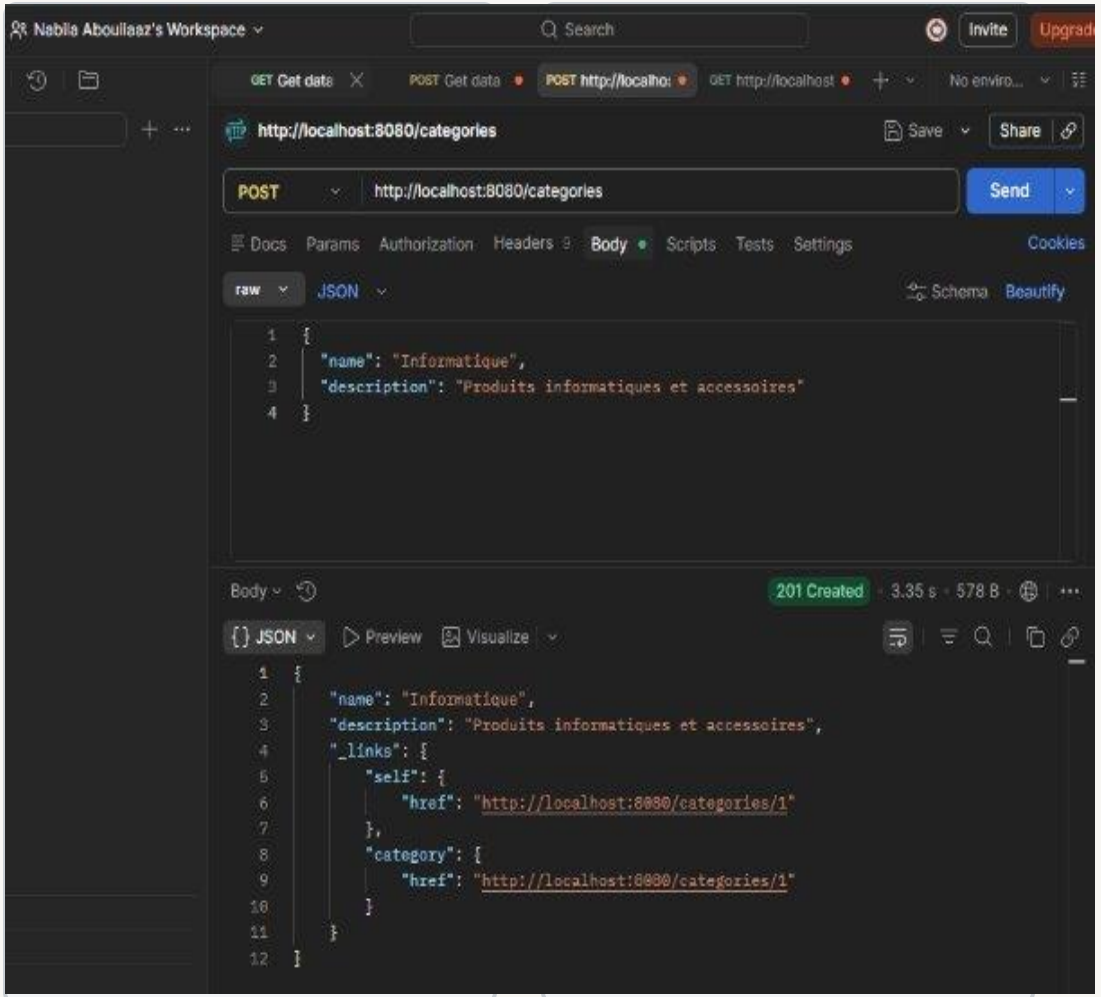
Principe

Grâce à @RepositoryRestResource, Spring Data REST génère automatiquement des endpoints pour Product et Category.

Scénario de test recommandé :

- 1. POST /categories : créer une catégorie
- 2. POST /products : créer un produit
- 3. PUT /products/{id}/category avec Content-Type: text/uri-list
- 4. GET /products/{id}/category : vérifier l'association

Une réponse 204 No Content lors du PUT confirme que la liaison a été enregistrée avec succès.



Difficultés rencontrées et solutions apportées

Configuration

Erreurs possibles dans application.properties, dépendances Maven ou choix du driver base de données.

Tests Postman

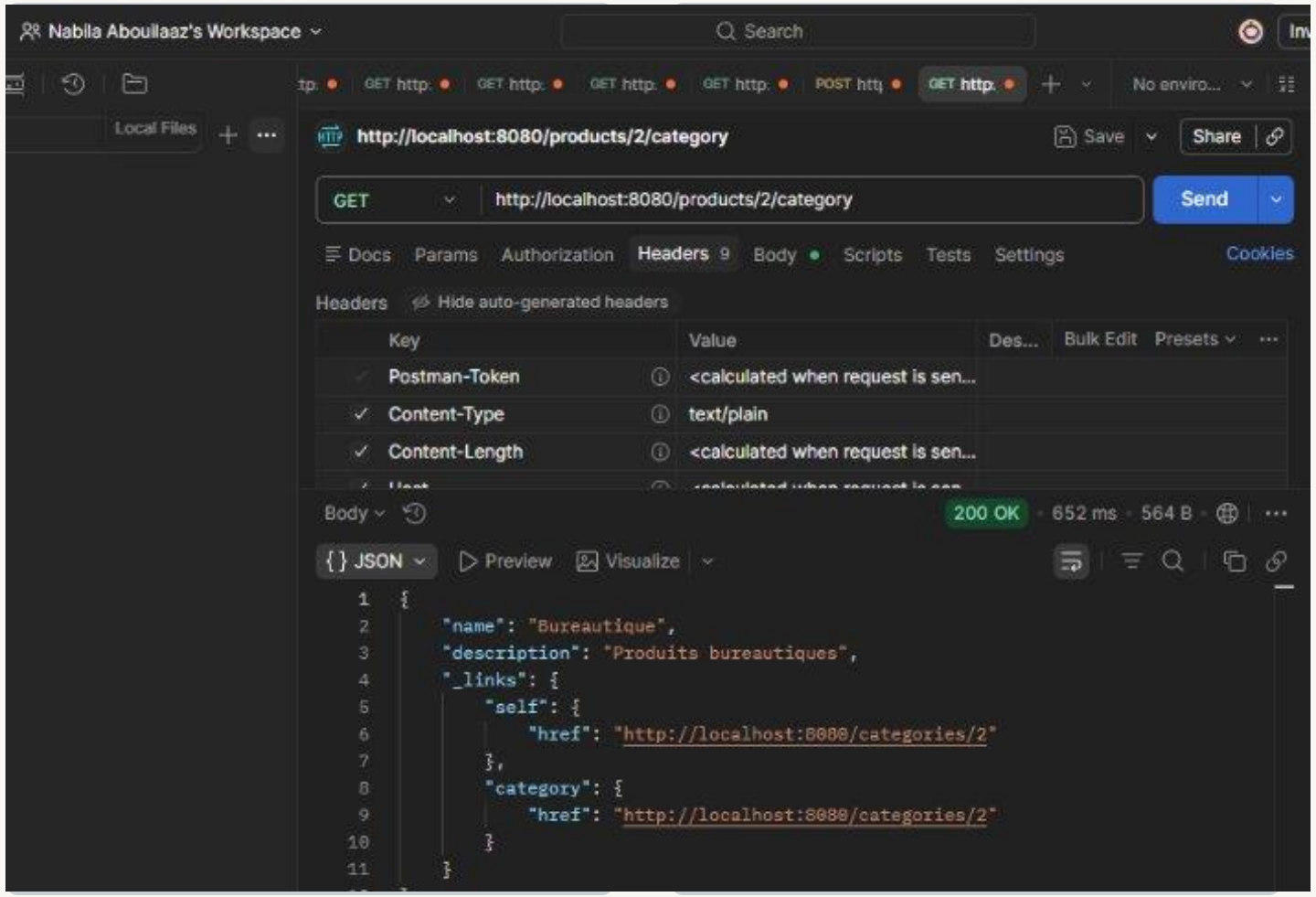
Erreurs 400 si le Body JSON est absent ; 404 si l'URL ou l'identifiant n'est pas correct.

Exercice Category

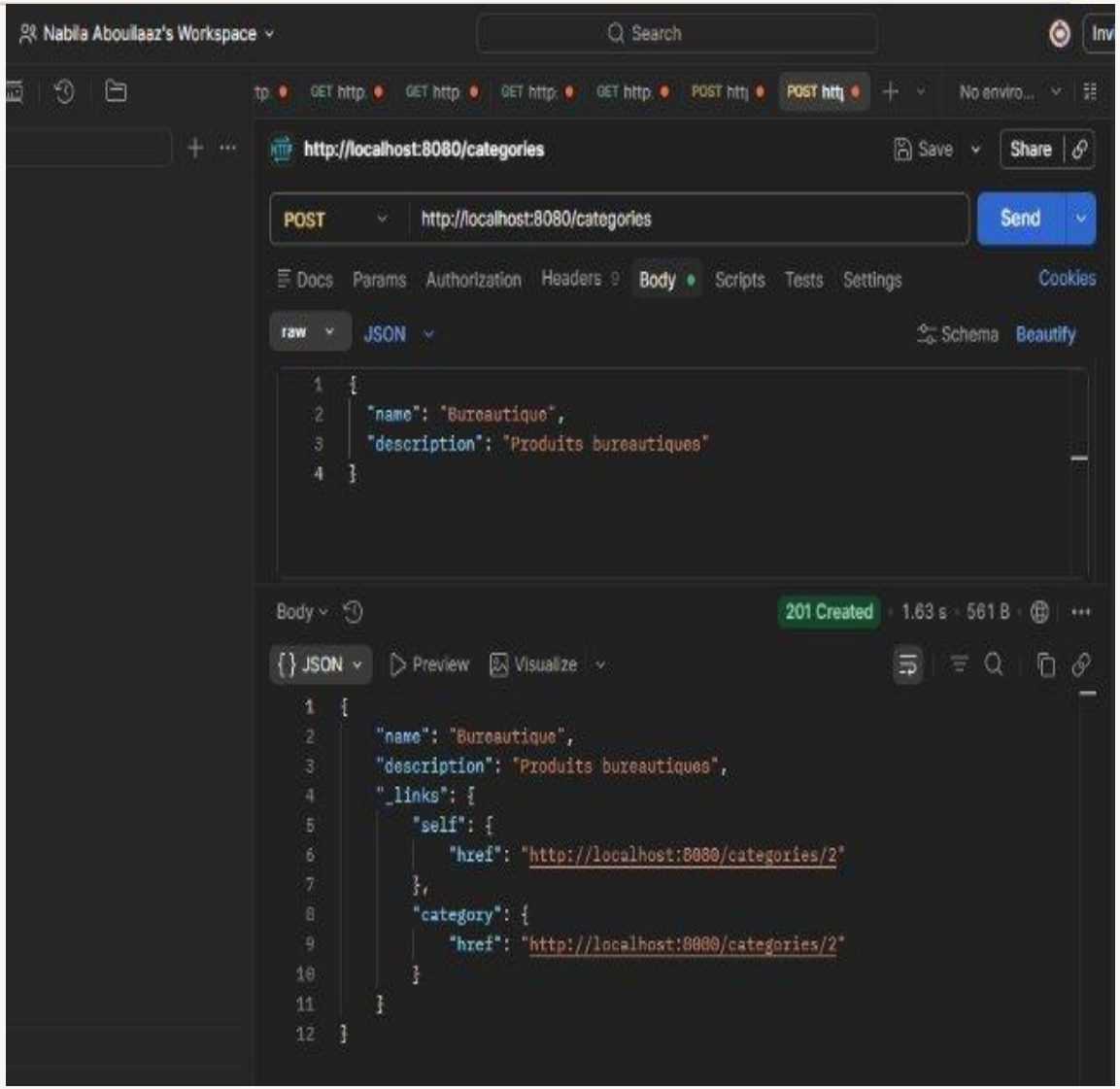
La liaison via Spring Data REST nécessite le bon header text/uri-list et une catégorie existante.

Bonne pratique

Toujours vérifier la réponse HTTP, les logs et les identifiants réellement créés.



Conclusion et éléments à compléter avant la remise

Ce TP m'a permis de comprendre le fonctionnement d'une application monolithique avec Spring Boot, la structuration en couches et la mise en place d'une API REST complète.

- CRUD validé sur l'entité Product
- Connexion à la base de données fonctionnelle
- Tests Postman réalisés et documentés
- Exercice Category / ManyToOne implémenté avec Spring Data REST